DEPARTAMENTO DE
INGENIERÍA
INFORMÁTICA
UNIVERSIDAD DE SANTIAGO DE CHILE

STRINGS

10110 – FUNDAMENTOS DE COMPUTACIÓN Y PROGRAMACIÓN

10145 - FUNDAMENTOS DE PROGRAMACIÓN PARA INGENIERÍA

RESUMEN DE CONTENIDOS

STRING

- En la primera clase del curso conocimos el tipo de dato `string`, y aprendimos que servían para almacenar símbolos o caracteres

- Es posible declararlos usando comillas simples (`'`) o dobles (`"`) y siempre se debe ser consistentes a la hora de **cerrar con el mismo tipo de comilla con el que se abrió** el string

Strings válidos:

```
"Hola mundo"
'Hola mundo'
"abc123"
```

Strings no válidos:

```
"Hola mundo'
'Hola mundo"
"Hola mundo"
```

STRING

- Dentro de un string se pueden usar comillas **del otro tipo**

```
>>> print("Un cuaderno de 'Google Colab'")
Un cuaderno de 'Google Colab'
```

- Para usar las mismas comillas se antepone el **carácter de escape** \

```
>>> print('Un ejemplo para \'comprender mejor\'')
Un ejemplo para 'comprender mejor'
```

- Las **triple comillas** definen un string de varias líneas: es el **docstring** que ya usamos para documentar funciones

OBJETO STRING

- Hasta aquí los datos eran valores sueltos: un `int` es solo un número
- Un string es, además, un **objeto**: un dato acompañado de dos cosas
  - **Atributos**: lo que guarda sobre sí mismo, sus caracteres y su orden
  - **Métodos**: las operaciones que sabe hacer con su propio contenido

- Un string sabe entregar su versión en mayúsculas, o responder si todos sus caracteres son dígitos
  - Un número no sabe hacer ninguna de las dos: sobre un número no tendrían sentido

  - Los métodos de los strings se revisan más adelante

OPERACIONES CON STRINGS

- Algunos operadores aritméticos permiten realizar nuevas operaciones al tratar con `strings`

- En particular, si existen `strings` involucrados, los operadores suma `(+)` y multiplicación `(*)` se vuelven el operador de **concatenación** y **repetición** respectivamente

```
>>> 'HOLA' + 'MUNDO'
'HOLAMUNDO'
>>> 'HOLA' * 3
'HOLAHOLAHOLA'
```

FUNCIONES DEL STRING

- Las funciones nativas `len()` y `str()` también trabajan con strings
  - `len(<string>)` entrega la cantidad de caracteres, espacios incluidos
  - `str(<expresión>)` entrega la expresión convertida en string

```
>>> print(len("Hola mundo"))
10
```

- El operador + necesita que **los dos operandos** sean strings

```
>>> 'hola' + 3
TypeError: can only concatenate str (not "int") to str
>>> print('hola' + str(3))
hola3
```

INDEXACIÓN

- Es posible acceder a un caracter en particular usando **indexación**: `<string>[posición]`

```
>>> texto = "HOLA MUNDO"
>>> print(texto[2])
L
```

- El primer caracter está en la posición `0` y el último en `len(<string>) - 1`
  - `texto[0]` es la H y `texto[9]` es la última O
  - Un índice fuera de ese rango entrega `IndexError: string index out of range`
  - Los índices negativos cuentan desde el final: `texto[-1]` es el último caracter

INMUTABILIDAD

- **Inmutabilidad**: una vez creado un string, sus caracteres no se pueden cambiar

```
>>> palabra = "Hola"
>>> palabra[0] = "B"
TypeError: 'str' object does not support item assignment
```

- **Lo que sí se puede hacer**: construir un string nuevo y reasignar la variable

```
>>> palabra = "B" + palabra[1:]
>>> print(palabra)
Bola
```

  - La variable pasa a apuntar a un string distinto: el original no se modificó
  - Por eso los métodos de los strings devuelven un string nuevo y dejan intacto el original

SLICING

- Slicing: extrae un trozo del string
  `<string>[inicio:fin:paso]`
  - `inicio`: primera posición que se incluye
  - `fin`: la posición en que se detiene, sin incluirla, como en `range()`
  - `paso`: de cuánto en cuánto avanza; es opcional y por defecto vale 1

```
>>> texto = "PROGRAMACION"
>>> print(texto[0:4], texto[4:])
PROG RAMACION
>>> print(texto[::-1])
NOICAMARGORP
```

  - Un índice fuera de rango da `IndexError`; un tramo fuera de rango no falla: `"Hola"[2:100]` entrega `"la"`

MÉTODOS DE LOS STRINGS

- Para utilizar un método de los `strings`, se debe utilizar un dato o variable que contenga un string, es decir, un **objeto string**
- A continuación **un punto** (.)
- Luego el **nombre del método**
- Finalmente **los parámetros que el método requiera**, entre paréntesis

```
<string>.<método>(<parámetros>)
```

MÉTODOS DE LOS STRINGS

- Un método se invoca **sobre el string mismo**, con un punto; una función recibe el dato entre paréntesis

```
>>> texto = "Hola mundo"
>>> print(texto.upper())
HOLA MUNDO
>>> print(len(texto))
10
```

  - `texto.upper()` se lee «pídele a `texto` que entregue su versión en mayúsculas»
  - Los métodos no modifican el string original: devuelven uno nuevo

MÉTODOS DE LOS STRINGS

- Los que entregan otro string:
  - `lower()` y `upper()`: copia en minúsculas o en mayúsculas
  - `title()`: copia con la inicial de cada palabra en mayúscula
  - `swapcase()`: copia con mayúsculas y minúsculas intercambiadas

- Los que responden `True` o `False`:
  - `islower()` y `isupper()`: si todas sus letras son de ese tipo
  - `isdigit()`: si todos sus caracteres son dígitos
  - `isalpha()`: si todos sus caracteres son letras

- **Para probar**: ¿qué responde `isdigit()` con `"-32"` o con `"3.1456"`? ¿Y `isalpha()` con tildes?

RECORRIDO DE STRINGS

- Recorrer un string: pasar por cada uno de sus caracteres

- Con for-in, cuando solo interesa el caracter

```
for caracter in texto:
  print(caracter)
```

- Con for y range, cuando además se necesita la posición

```
for i in range(len(texto)):
  print(i, texto[i])
```

- Con while, cuando el recorrido puede detenerse antes del final

```
i = 0
while i < len(texto):
  print(texto[i])
  i = i + 1
```

- El identificador nombra la unidad del recorrido: al recorrer un string siempre es un caracter, aunque se le llame `palabra`

EJERCICIOS

EJERCICIO PROPUESTO 1

- Pongamos en práctica lo que vimos sobre indexación y slicing:

Escriba un programa que pida una palabra al usuario y muestre esa misma palabra escrita al revés. Luego, usando ese resultado, el programa debe indicar si la palabra es un palíndromo, es decir, si se lee igual en los dos sentidos.

EJERCICIO PROPUESTO 2

- Utilicemos el recorrido de strings para resolver el siguiente problema:

"Escriba un programa que pida una frase al usuario y cuente cuántas veces aparece la letra a en ella, sin distinguir mayúsculas de minúsculas. El programa debe informar el resultado con un mensaje al usuario"